

Hierarchical Matching: An Efficient Knowledge Graph Retrieval Method for Retrieval-Augmented Generation

Yu Li^{a, b, *}, QuanSheng Bai^a, Zhijin Li^a, Keqin Liu^a, Chang Shu^a, Huaili Peng^c

a. Technical Research Center, China Datang Digital Technology Co., Ltd, Xiong'an, Hebei 070001, PR China

b. State Key Laboratory of Advanced Design and Manufacturing Technology for Vehicle, Hunan University, Changsha, Hunan 410082, PR China

c. Beijing SAWES Energy Technology Co., Ltd. Changsha Branch, Changsha, Hunan 410023, PR China

Abstract

Knowledge Graphs (KGs) are commonly used in Retrieval-Augmented Generation (RAG) systems to provide structured and easily accessible information. However, there are several limitations to using KGs in RAG, including low efficiency, poor data quality, and information overload. This study introduces a novel KG retrieval technique called the Hierarchical Matching Strategy (HMS), which employs a multi-level matching technique. The HMS-based RAG system achieves efficiency comparable to that of commercial platforms. Testing results indicate that this approach demonstrates satisfactory performance for RAG-based intelligent assistants.

Keywords: knowledge graph; retrieval-augmented generation; hierarchical matching strategy; large language models; question-and-answer

1. Introduction

A Knowledge Graph (KG) [1] is a structured data model representing entities and their relationships in the real world. Since Google introduced the concept in 2012 [2], KGs have become essential across various fields. The primary goal of KGs is to enhance search engines by integrating structured entity information with unstructured data. This integration allows search engines to better understand user queries and deliver more relevant responses.

* Corresponding author, Y. Li, E-mail yli03191990@gmail.com, Tel 86-0731-88232831

At the core of KGs is the representation of data in graph form, where nodes denote entities (such as people, places, objects, etc.), and edges indicate the relationships between these entities (such as "born in," "located in," etc.). This structured approach enables the efficient storage, management, and querying of knowledge across different application scenarios. Unlike traditional databases, KGs facilitate not only basic information retrieval but also complex reasoning and semantic understanding.

KGs have proven to be versatile tools across various domains, offering structured representations of knowledge through entities and relationships. They are employed in numerous applications, including search engines, recommendation systems, and Natural Language Processing (NLP). In search engines, KGs enhance search results by providing structured information, enabling more accurate and contextually relevant responses. Features like rich snippets and answer boxes are improved, thereby enhancing the user experience [3]. In recommendation systems, KGs leverage the relationships between entities and user interactions to offer more personalized and diverse recommendations [4, 5]. Moreover, KGs support various NLP tasks, including Question-and-Answer (QA) systems and sentiment analysis. By providing context and background knowledge, they enable better comprehension and generation of human language [6]. In the healthcare sector, KGs integrate diverse biomedical data, assisting in drug discovery, disease diagnosis, and personalized medicine. They represent complex biological entities and their interactions, facilitating advanced analytical tasks [7].

Recently, Retrieval-Augmented Generation (RAG) [8], a core technology in Generative Artificial Intelligence (GAI), has brought new possibilities for the applications of KGs. RAG leverages the strengths of Large Language Models (LLMs) [9] and external knowledge sources to enhance the accuracy and relevance of generated content. In a RAG system, a retriever model first selects relevant pieces of information from a large database or document set. This retrieved information is then provided to a generator model, usually a large language model, which uses the context to produce a

more informed and accurate response. KGs play a crucial role in RAG systems by offering structured and easily accessible information [8]. They enhance the retriever model's capability to identify relevant facts and relationships, enabling the generator to create more contextually accurate and comprehensive responses. By integrating KGs into RAG, the system can utilize a rich repository of structured knowledge, thereby improving the quality and reliability of the generated outputs.

Despite these advantages, there are several limitations associated with the use of KGs in RAG [10]:

- **Low efficiency.** As the size of KGs grows, the efficiency of retrieving relevant information diminishes. Large KGs require more computational resources for storage, indexing, and querying, which can lead to increased latency in real-time applications.
- **Bad quality of Data.** The accuracy of information retrieved from KGs heavily depends on the quality and completeness of the data. Incomplete or incorrect data entries can lead to inaccurate retrieval results, affecting the reliability of the information.
- **Information Overload.** KGs can return a vast amount of related information for a given query, especially when dealing with popular or well-connected entities. This over-retrieval can overwhelm users with excessive data, making it challenging to identify the most relevant facts.

To address these limitations, this study proposes a novel method called the Hierarchical Matching Strategy (HMS) for retrieving relevant knowledge from KGs, specifically tailored for integration into the RAG framework. The method retrieves related knowledge for a given query through natural language similarity and uses multi-threaded parallel computing to improve retrieval efficiency. Additionally, it can be effectively used to establish intelligent QA assistants for enterprise systems, addressing problems such as data silos and knowledge dispersion in asset-intensive, technology-intensive, and knowledge-intensive enterprises.

The subsequent sections of this study are organized as follows: Section 2 introduces the KG with extracted expert knowledge. Section 3 details the framework for the proposed HMS and presents an RAG-based intelligent QA assistant. In Section

4, the feasibility of the proposed method is demonstrated using different questions. Finally, the concluding section offers prospective remarks.

2. Knowledge Mining for Knowledge Graphs

Neo4j [11] is a popular graph database management system that uses a graph data model to represent and store data. Unlike traditional Relational Databases (RDBs), which organize data in tables and rows, Neo4j represents data as nodes and relationships, allowing for highly efficient querying and traversal of complex networks. Its query language, Cypher, is a descriptive, declarative graph query language designed for graph analysis, known for its expressive power, user-friendliness, and ease of use.

In this study, Neo4j is used to construct a company-level KG that stores various regulation documents. As illustrated in Fig. 1, 117 regulations are stored across three layers: the first layer represents the company (marked in red), which can further store other categories of knowledge; the second layer consists of regulation names (marked in blue); and the third layer contains institutional provisions (marked in grey). To facilitate retrieval and matching, the nodes in these three layers are labeled as "company," "regulation," and "regulation sub-item," respectively. Statistically, the KG comprises 117 nodes labeled as "regulation," 3,903 nodes labeled as "regulation sub-item," and 6,019 relationships.

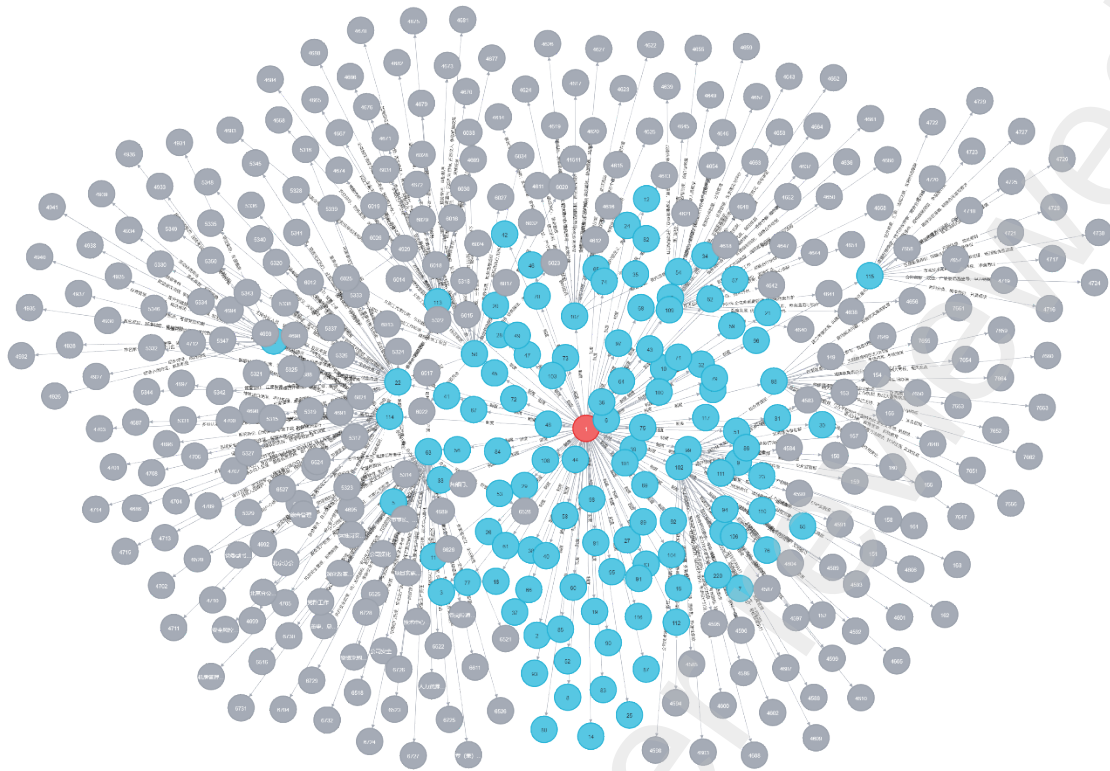


Fig. 1. The KG storing regulation documents.

To ensure data integrity, traditional document splitting methods based on fixed chunk sizes are not used. Instead, documents are split according to paragraphs. Additionally, for regulation-related documents, a query may span multiple paragraphs. Therefore, if a paragraph begins with a list marker, it is merged with the preceding paragraph to form a single unit. This approach might result in a node containing a large amount of content, potentially exceeding the maximum token length for embedding models. This issue can be effectively managed by defining relationships that summarize the content of the connected node, labeled with the corresponding regulation sub-item. By calculating the similarity between the query and these relationships, the problem of content length can be significantly mitigated.

Furthermore, the KG primarily stores text data, but regulation documents often contain a large number of images, tables, and attachments. Fortunately, in Neo4j, each node can be instantiated and can support additional attributes. Based on this capability, each regulation sub-item node has been assigned "images" and "appendix" attributes, which store images, tables, templates, and attachments from the regulations. When querying regulation-related questions, the corresponding images and appendix can be

accessed. This approach effectively addresses the challenge of handling multimodal data in KG-based RAG.

3. The Proposed Hierarchical Matching Strategy

3.1. Problem Analysis

LLMs are trained on extensive datasets that include dialogue data between natural language and Cypher, enabling accurate mapping of queries to corresponding Cypher queries, a process known as Text-to-Cypher (Text2Cypher). However, unlike RBDs, KGs have more complex structures with a greater variety of relationships and attribute fields. Theoretically, with n ($n \gg 1,000$) relationships, there could be n different relationships, resulting in lower accuracy for Text2Cypher. Therefore, the Text2Cypher technique is not suitable for use in RAG.

Similarity calculations using embeddings can evaluate the correlation between a query and knowledge. Embedding is a Machine Learning (ML) and NLP technique that converts high-dimensional data, such as words or phrases, into lower-dimensional vectors. These vectors, often called embeddings, capture the semantic meaning and relationships of the data in a continuous vector space. The primary goal of embeddings is to transform categorical data, typically sparse and high-dimensional, into dense, real-valued vectors that can be easily processed by machine learning algorithms.

For a KG containing 4,021 nodes and 6,019 relationships, individually calculating similarities for all nodes and relationships is time-consuming, as shown in Fig. 2. Loading the nodes and relationships from the KG takes over 25 seconds, while the embedding process requires more than 85 seconds. Notably, if the embeddings are further used to compute similarities, such as cosine similarity, the total time required would effectively double. This substantial time cost does not meet the efficiency requirements for intelligent assistants.

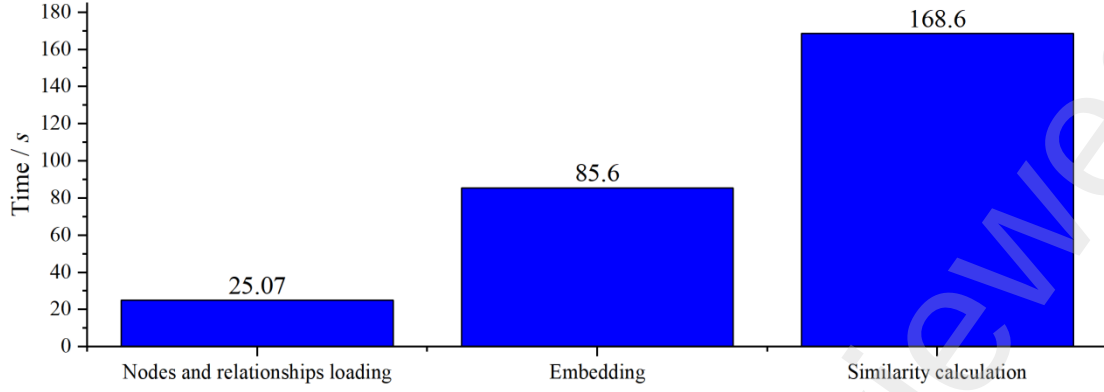


Fig. 2. Efficiency evaluation of a KG.

3.2. Hierarchical Matching Strategy

In this study, Bilingual and Crosslingual Embedding (BCEmbedding) model, developed by NetEase Youdao [12], including the EmbeddingModel and RerankerModel, is employed. The EmbeddingModel specializes in generating semantic vectors for semantic search and QA, while the RerankerModel excels at refining search results and ranking tasks, efficiently addressing the issue of information overload mentioned in Section 1.

Cosine similarity [13] measures the directional similarity of embeddings and is calculated as follows:

$$s(\mathbf{q}, \mathbf{d}) = \frac{\text{sum}(\mathbf{q} \cdot \mathbf{d})}{\|\mathbf{q}\| \cdot \|\mathbf{d}\|} = \frac{\sum_i (q_i \cdot d_i)}{\sqrt{\sum_i q_i^2} \cdot \sqrt{\sum_i d_i^2}} \quad (2)$$

where $\mathbf{q}$ and $\mathbf{d}$ are embeddings of query and document, respectively; and q_i and d_i are their elements.

However, relying solely on embedding similarity can result in losing significant information. Therefore, the Maximal Marginal Relevance (MMR) [14] is employed to balance the similarity between query and document and the diversity of documents.

$$\text{MMR}(\mathbf{q}, \mathbf{d}_i) = \arg \max_{d_i \in D} \left[\lambda \cdot s(\mathbf{q}, \mathbf{d}_i) - (1 - \lambda) \cdot \max_{d_j \in R} s(\mathbf{d}_i, \mathbf{d}_j) \right] \quad (3)$$

where λ is a balance parameter; and s is calculated using cosine similarity.

The HMS comprises two stages.

3.2.1. Stage I. Retrieve knowledge corresponding to the query

HMS organizes the matching process into multiple levels, corresponding to different labels of nodes in the KG. This stage includes two steps: low-level attribute matching and high-level relationship matching. High-level matching builds upon low-level matching, progressively narrowing down potential matches to enhance both efficiency and accuracy.

Step 1. Low-level attribute matching. HMS first focuses on low-level attributes, such as detailed rules and attributes, which are fewer in number than attribute relationships. Nodes labeled as "regulation sub-item" are matched using the Cypher query "MATCH (n: regulation sub-item) RETURN n". Then, The similarity between each node and the input query is calculated, and if the similarity exceeds a defined threshold δ , the corresponding knowledge chains are collected.

Step 2. High-level relationship matching. For attributes with a small δ in Step 1, relationships are matched using the Cypher query "MATCH (n) -[r]-> (m {name: 'regulation sub-item content'}) RETURN r". The similarity between each relationship and the input query is calculated, and if the similarity exceeds δ , the corresponding knowledge chains are collected.

The threshold δ is crucial. A large δ can address information overload but may miss relevant knowledge, leading to inefficiency. Conversely, a small δ ensures the retrieval of relevant knowledge but may cause information overload. Thanks to the RerankerModel, redundant information can be further reranked and filtered. Therefore, this study uses a small δ .

3.2.2. Stage II. Rerank the retrieved knowledge corresponding to the query

After retrieving knowledge, the RerankerModel is employed to reorder or rescore the initial candidate list generated by the primary retrieval model, leading to more accurate and contextually appropriate results. After an initial retrieval phase, RerankerModel refines the ranking of documents by comparing the cross-lingual embeddings of the query and the retrieved documents, allowing for a more accurate ranking based on the semantic similarity across languages.

For a given query, the model retrieves an initial set of candidate documents and computes the similarity score for each document:

$$\text{Score}(\mathbf{q}, \mathbf{d}_i) = s(\mathbf{q}, \mathbf{d}_i) \quad (4)$$

where d_i represents the i -th document in the candidate set. The documents are then reranked based on these scores, with higher scores indicating more relevant documents.

The RerankerModel is trained to maximize the similarity score for relevant query-document pairs while minimizing it for irrelevant pairs, often using a margin-based loss function like the hinge loss:

$$L = \sum_{(q, d^+, d^-)} \max(0, \text{margin} - s(\mathbf{q}, \mathbf{d}^+) + s(\mathbf{q}, \mathbf{d}^-)) \quad (5)$$

where $\mathbf{d}^+$ and $\mathbf{d}^-$ represent a relevant and an irrelevant document, respectively, for query q . The model is trained to ensure that the similarity score for a relevant document is higher than that for an irrelevant document by at least a certain margin.

3.2.3. The hierarchical matching strategy architecture

The architecture of the proposed HMS is shown in Fig. 3. Stage I can be further accelerated by over 50% using multithreading. To enable LLMs to better understand the graph structure of retrieved knowledge chains, these chains are stored in the format “entity >> relationship >> attribute”.

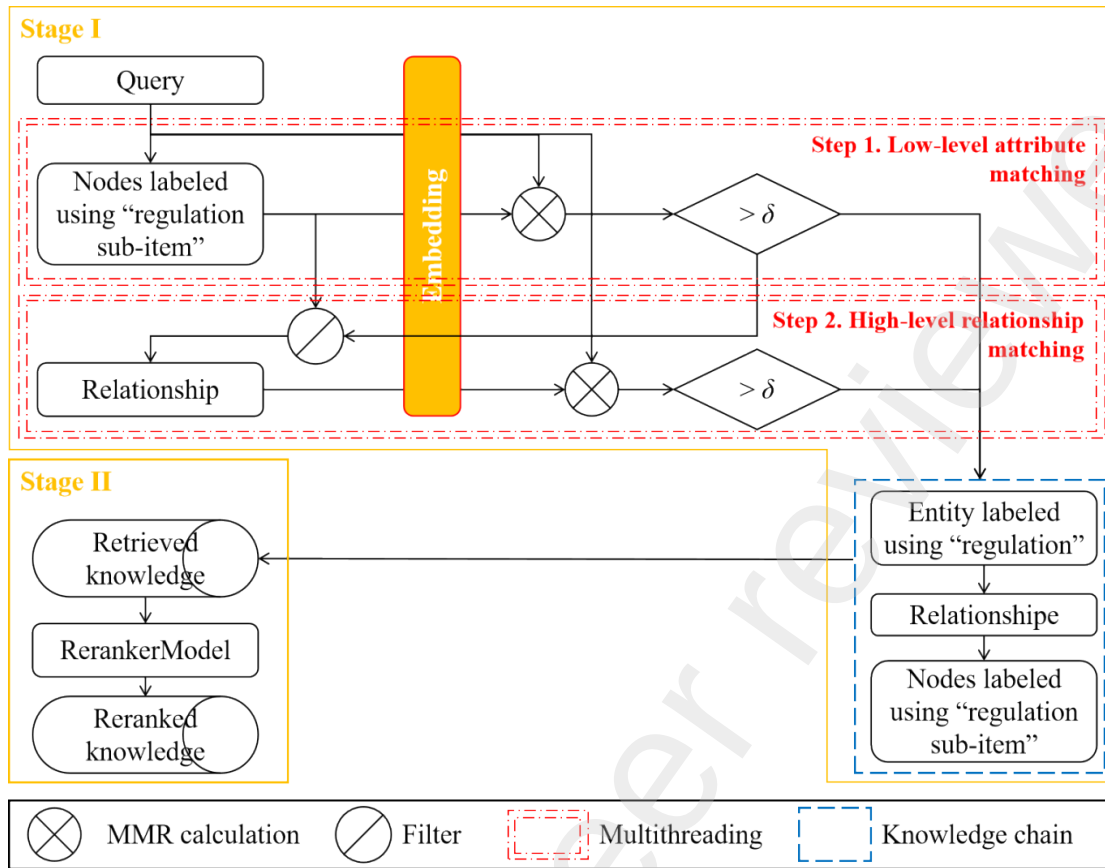


Fig. 3. The HMS architecture.

3.3. Retrieval-Augmented Generation-based Intelligent Assistant

As shown Fig. 4, the proposed HMS is employed to establish an RAG as follows:

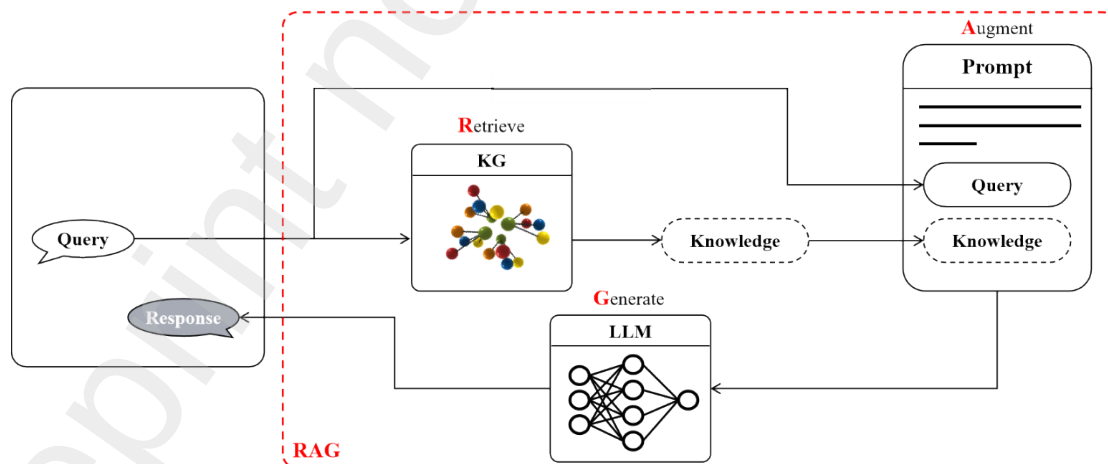


Fig. 4. The KG-based RAG.

Step 1. HMS retrieves the related knowledge from the KG corresponding to the input query.

Step 2. A prompt is constructed to allow the LLM to summarize and infer answers from the knowledge.

Step 3. The constructed prompt is fed to the LLM, and the final response corresponding to the input query is obtained.

In the RAG system, the LLM summarizes responses from the retrieved knowledge. Although the issue of information overload can be efficiently managed by defining a large δ and using the RerankerModel, the amount of relevant knowledge may increase with the growth of the KG. This phenomenon requires a strong Natural Language Understanding (NLU) capability in LLMs to comprehend long texts. Currently, various LLMs have been open-sourced with different architectures, parameter scales, training strategies, and hyperparameter settings, leading to varying capabilities in long-text understanding tasks. In this study, various LLMs with parameter scales under 10B are employed for performance comparison.

- ChatGLM3-6B [15], developed by Zhipu AI, is trained on approximately 1TB of bilingual Chinese and English data, with optimizations for Chinese QA and dialogue through supervised fine-tuning and reinforcement learning.
- LLaMA2-7B [16] is an open-source model by Meta. It is trained on 2 trillion tokens with a context support length of 4,096.
- LLaMA2-Chinese-7B is fine-tuned on a Chinese dataset based on LLaMA2, offering enhanced capabilities in Chinese dialogue.
- Qwen-7B [17] is an open-source model by Alibaba, and trained on large-scale, high-quality, and diverse data covering both Chinese and English. Qwen-7B has demonstrated strong competitive performance in relevant benchmarks.
- LLaMA3-8B [16] is a new model introduced by Meta, building on LLaMA2, with a new tokenizer and expanded vocabulary size for more efficient text encoding.

In this section, 42 random questions are employed. Each question's retrieval includes varying numbers of knowledge chains, ranging from several to dozens. As shown in Fig. 5, the generated responses are scored as follows: 100 points for correct

answers; 80 points for correct answers containing some irrelevant information; 70 points for answers in English or correct reasoning but incorrect summarization; 60 points for correct answers with excessive irrelevant information; 50 points for incomplete answers; and 0 points for incorrect answers.

问题	ChatGLM3-6B	LLaMA2-7B	LLaMA2-Chinese-7B	Qwen-7B	LLaMA3-8B
Q1	100	80	0	100	100
Q2	80	70	80	60	100
Q3	60	60	0	100	100
Q4	100	100	0	100	100
Q5	100	80	100	100	100
Q6	0	60	60	60	100
Q7	0	70	0	0	100
Q8	60	70	0	0	100
Q9	0	0	0	0	70
Q10	0	70	0	0	100
Q11	100	70	0	100	100
Q12	100	0	100	100	100
Q13	100	100	100	100	100
Q14	100	70	0	100	100
Q15	0	0	0	0	0
Q16	100	100	100	100	100
Q17	100	80	50	100	50
Q18	100	0	0	0	100
Q19	100	0	0	50	100
Q20	100	70	50	80	100
Q21	100	50	0	50	0
Q22	100	70	0	100	80
Q23	80	70	0	100	100
Q24	100	100	100	100	100
Q25	50	50	50	50	50
Q26	0	0	0	0	0
Q27	0	0	0	0	100
Q28	100	0	0	60	100
Q29	60	70	0	0	80
Q30	100	70	0	100	80
Q31	100	70	100	0	0
Q32	100	0	0	0	0
Q33	0	100	0	100	100
Q34	100	80	100	60	50
Q35	0	0	0	100	100
Q36	50	0	0	0	50
Q37	100	70	100	100	100
Q38	100	70	0	50	100
Q39	50	70	50	50	100
Q40	50	0	0	80	100
Q41	60	70	100	50	100
Q42	100	0	0	100	100
标准差	40.4131	36.6640	42.3085	42.3085	33.9442
均值	69.0476	51.4286	29.5238	59.5238	81.1905

Fig. 5. LLM comparisons.

The best performance is observed with LLaMA3-8B, which has the highest average score and the smallest standard deviation, followed by ChatGLM3-6B. LLaMA2-7B and Qwen-7B have similar scores, but LLaMA2-7B has a smaller standard deviation. Notably, LLaMA2-7B is less suited for Chinese, often responding in English despite the constraint in the prompt that responses should be in Simplified

Chinese. Although LLaMA2-Chinese-7B is further trained on Chinese data, its performance in NLU is unsatisfactory. Therefore, this study employs LLaMA3-8B in the RAG system.

4. Experiments and Results Analysis

QA is a specialized area within the field of Information Retrieval (IR) [18]. In this section, the efficiency and accuracy of the proposed method is evaluated.

4.1. Efficiency and Accuracy Analysis

The retrieval process of the HMS includes four main components: attribute (nodes labeled with regulation sub-item) matching (t_1), relationship matching (t_2), format knowledge chains (t_3), and reranking (t_4). As shown in Table 1, ten queries are employed to evaluate the retrieval efficiency of the HMS. The total time (Σt) is not the sum of these four components but represents the overall duration of the entire search process, including environment loading and LLM answering.

It is observed that the majority of the time is spent on t_1 and t_2 , accounting for 40% to 60% of the total duration. The maximum value of Σt is 6.086 seconds, with an average of 4.625 seconds and a standard deviation of 1.047 seconds. It is expected that Σt can be further reduced after deploying HMS in products as a service. As indicated in Table 2, the response time of existing commercial LLMs typically ranges from 2 to 5 seconds. Thus, the efficiency of the proposed HMS is comparable to the average performance level of commercial platforms.

Table 1 Efficiency evaluation for the HMS.

Query	t_1/s	t_2/s	t_3/s	t_4/s	$\Sigma t/s$
Q1	0.627	1.68	0.005	0.313	3.455
Q2	0.692	1.669	0.006	0.323	4.248
Q3	0.724	1.716	0.047	0.305	5.401
Q4	0.659	1.743	0.003	0.316	3.284
Q5	0.704	1.651	0.004	0.326	3.287
Q6	0.611	1.537	0.004	0.309	5.051
Q7	0.721	1.931	0.006	0.397	5.859
Q8	0.633	2.041	0.007	0.363	5.779

Q9	0.638	1.472	0.004	0.313	3.657
Q10	0.667	1.754	0.004	0.314	3.729
STD	0.041	0.168	0.013	0.029	1.05
Mean	0.668	1.719	0.009	0.328	4.375

Table 2 QA response time of commercial LLMs.

LLM	ChatGPT [19]	GLM-4	Qwen2	Gemini [20]	Xinghuo
Time / s	2-4	3-5	3-5	2-4	3-5

As previously mentioned, pre-trained LLMs can perform Text2Cypher tasks using prompt engineering without the need for fine-tuning. In this study, LLaMA3-8B-based Text2Cypher is tested. Prompt engineering is defined as follows:

'''

You are an expert in knowledge graphs, proficient in retrieving and querying using Neo4j. Given the question {input query}, create a syntactically correct Cypher query. The Neo4j database includes the following nodes and relationships: {nodes and relationships information}.

'''

Here, node properties (including labels, names, and IDs) and relationships are provided as part of the nodes and relationships information.

For a given query, "Which department is responsible for the establishment of systems such as ISO9001 and CMMI5?", the generated Cypher query using Text2Cypher is:

'''

```
MATCH k=(n1: regulation)-[*]->(n2)
WHERE n2.name CONTAINS 'ISO9001 and CMMI5'
RETURN k
```

'''

When this Cypher query is executed in Neo4j, the retrieved knowledge chain is empty. This is because the relevant content in the KG is described as "ISO9001 and CMMI 5." The difference in spacing between "CMMI 5" and "CMMI5" caused the failure to match the relevant knowledge. This demonstrates the challenge of achieving high accuracy with Text2Cypher, due to the diversity and complexity of relationships

and attributes, as discussed in Section 3.1.

Based on the proposed HMS technique, the relevant knowledge for the input query can be accurately matched: “Project Matrix Management Work Norms >> Responsibilities of the Project Management Department >> ... Carry out the construction of systems such as ISO9001 and CMMI 5 ...”.

This demonstrates that Text2Cypher is not well-suited for use in RAG due to its limitations in handling variations in textual data. On the other hand, the HMS technique, with a multi-level retrieval technique, effectively supports RAG by ensuring accurate and comprehensive retrieval of relevant knowledge.

4.2. Response Quality Analysis

RAG Answer Quality (RAGAS) [21] is a framework designed for reference-free evaluation of RAG systems. It comprehensively assesses RAG performance based on two main criteria: retrieval quality and generation quality. RAGAS evaluates retrieval quality through context relevance, and generation quality through answer faithfulness and answer relevance.

Context relevance measures the retrieval quality by determining the proportion of retrieved knowledge that is useful for generating the answer. It is specifically defined as:

$$CR = \frac{n_a}{n_t} \quad (6)$$

where n_a is the number of sentences in the retrieved knowledge that are relevant to the answer; and n_t is the total number of sentences in the retrieved knowledge.

Answer faithfulness assesses the consistency and accuracy of the generated content in relation to the original content. In this study, the Facebook Natural Language Inference (NLI) open-source model, bart-large-mnli [22, 23], is used. If the generated text is classified by the model as "Entailment" or "Neutral," the answer's faithfulness is considered high; and if classified as "Contradiction," the faithfulness is considered low.

Answer relevance evaluates the quality of the generated answer by calculating the

similarity between the answer and the question. It is defined as:

$$AR = \cos(\mathbf{q}, \mathbf{a}) = \frac{\sum_i (q_i \times a_i)}{\sqrt{\sum_i q_i^2} \times \sqrt{\sum_i a_i^2}} \quad (7)$$

where q_i and a_i are the word vectors representing the question and the answer, respectively.

The RAGAS results for 10 randomly selected queries are shown in Table 3. The average context relevance score is 0.105, indicating that there is room for improvement in retrieving relevant knowledge. The scores for answer faithfulness and answer relevance are notably high, with all answers categorized under “Entailment” for answer faithfulness. This suggests that the RAG system generates answers that are not only relevant but also faithful to the source information. Furthermore, the low standard deviation reflects the system's good stability and consistent performance across different queries.

Table 3 RAGAS results.

Query	Context relevance	Answer faithfulness	Answer relevance
Q1	0.074	0.83 (Entailment)	0.83
Q2	0.066	0.77 (Entailment)	0.94
Q3	0.090	0.97 (Entailment)	0.88
Q4	0.160	0.96 (Entailment)	0.87
Q5	0.167	0.96 (Entailment)	0.87
Q6	0.100	0.85 (Entailment)	0.90
Q7	0.017	0.85 (Entailment)	0.89
Q8	0.167	0.79 (Entailment)	0.89
Q9	0.081	0.88 (Entailment)	0.91
Q10	0.13	0.92 (Entailment)	0.88
STD	0.049	0.07	0.03
Mean	0.105	0.88	0.89

Conclusions

In this study, a novel KG retrieval technique named HMS is introduced. Additionally, an HMS-based RAG system is developed as an intelligent assistant for regulatory QA. The key contributions are summarized as follows:

1. The multi-level matching approach of the proposed HMS demonstrates

efficiency comparable to that of commercial platforms. Importantly, QA time can be further reduced if HMS is deployed as a service.

2. Pre-trained LLMs exhibit low accuracy in the Text2Cypher task, making it challenging to retrieve relevant information from KGs in RAG systems. In contrast, the proposed HMS can accurately match knowledge related to the input query in a KG.

3. Tests with various queries show that an external database can effectively assist LLMs in acquiring domain-specific knowledge. This approach significantly addresses inherent weaknesses in LLMs, such as hallucinations and lack of expertise.

Acknowledgments

The authors would like to express their sincere gratitude to Datang Xianyi Technology Co., Ltd for providing the necessary resources, including access to proprietary software and data, which were crucial for conducting this research.

References

- [1] Pu, H., T. Hu, T. Song, P. Schonfeld, X. Wan, W. Li, and L. Peng, *Modeling and application of a customized knowledge graph for railway alignment optimization*. Expert Systems with Applications, 2024. **244**: p. 122999.
- [2] Fensel, D., U. Şimşek, K. Angele, E. Huaman, E. Kärle, O. Panasiuk, I. Toma, J. Umbrich, A. Wahler, and D. Fensel, *Introduction: what is a knowledge graph?* Knowledge graphs: Methodology, tools selected use cases, 2020: p. 1-10.
- [3] Mai, G., K. Janowicz, and B. Yan. *Combining Text Embedding and Knowledge Graph Embedding Techniques for Academic Search Engines*. in *Semdeep/NLIWoD@ ISWC*. 2018.
- [4] Yang, X., Z. Huan, Y. Zhai, and T. Lin, *Research of personalized recommendation technology based on knowledge graphs*. Applied Sciences, 2021. **11**(15): p. 7104.
- [5] Shao, B., X. Li, and G. Bian, *A survey of research hotspots and frontier trends of recommendation systems from the perspective of knowledge graph*. Expert Systems with Applications, 2021. **165**: p. 113764.
- [6] Costello, J., M. Kaur, M.Z. Reformat, and F.V. Bolduc, *Leveraging knowledge graphs and natural language processing for automated web resource labeling and knowledge mobilization in neurodevelopmental disorders: development and usability study*. Journal of Medical Internet Research, 2023. **25**: p. e45268.
- [7] Peng, C., F. Xia, M. Naseriparsa, and F. Osborne, *Knowledge graphs: Opportunities and challenges*. Artificial Intelligence Review, 2023. **56**(11): p. 13071-13102.
- [8] Lewis, P., E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, and T. Rocktäschel, *Retrieval-augmented generation for knowledge-intensive nlp tasks*. Advances in Neural Information Processing Systems, 2020. **33**: p. 9459-9474.

- [9] Baghadasaryan, A., T. Bunarjyan, A. Poghosyan, A. Harutyunyan, and J. El-Zein, *Knowledge retrieval and diagnostics in cloud services with large language models*. Expert Systems with Applications, 2024: p. 124736.
- [10] Paulheim, H., *Knowledge graph refinement: A survey of approaches and evaluation methods*. Semantic web, 2017. **8**(3): p. 489-508.
- [11] Fernandes, D. and J. Bernardino, *Graph Databases Comparison: AllegroGraph, ArangoDB, InfiniteGraph, Neo4J, and OrientDB*. Data, 2018. **10**: p. 0006910203730380.
- [12] NetEase Youdao, I. *BCEmbedding: Bilingual and Crosslingual Embedding for RAG*. 2023; Available from: <https://github.com/netease-youdao/BCEmbedding>.
- [13] Thongtan, T. and T. Phienthrakul. *Sentiment classification using document embeddings trained with cosine similarity*. in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: Student Research Workshop*. 2019.
- [14] Gunawan, D., S.H. Harahap, and R.F. Rahmat. *Multi-document summarization by using textrank and maximal marginal relevance for text in Bahasa Indonesia*. in *2019 International conference on ICT for smart society (ICISS)*. 2019. IEEE.
- [15] Du, Z., Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, *Glm: General language model pretraining with autoregressive blank infilling*. arXiv preprint arXiv:10360, 2021.
- [16] Touvron, H., L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, and S. Bhosale, *Llama 2: Open foundation and fine-tuned chat models*. arXiv preprint arXiv:09288, 2023.
- [17] Bai, J., S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, *Qwen-vl: A frontier large vision-language model with versatile abilities*. arXiv preprint arXiv:12966, 2023.
- [18] Allam, A.M.N. and M.H. Haggag, *The question answering systems: A survey*. International Journal of Research and Reviews in Information Sciences (IJRRIS), 2012. **2**(3).
- [19] Bhattacharya, P., V.K. Prasad, A. Verma, D. Gupta, A. Sapsomboon, W. Viriyasitavat, and G. Dhiman, *Demystifying ChatGPT: An In-depth Survey of OpenAI's Robust Large Language Models*. Archives of Computational Methods in Engineering, 2024: p. 1-44.
- [20] Gozalo-Brizuela, R. and E.C. Garrido-Merchan, *ChatGPT is not all you need. A State of the Art Review of large Generative AI models*. arXiv preprint arXiv:04655, 2023.
- [21] Es, S., J. James, L. Espinosa-Anke, and S. Schockaert, *Ragas: Automated evaluation of retrieval augmented generation*. arXiv preprint arXiv:15217, 2023.
- [22] Yin, W., J. Hay, and D. Roth, *Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach*. arXiv preprint arXiv:00161, 2019.
- [23] Lewis, M., Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, *Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension*. arXiv preprint arXiv:13461, 2019.